

```
[livecd /]# cp /etc/resolv.conf /mnt/gentoo/etc/
```

Next, mount `/dev` and `/proc`:

```
[livecd /]# mount -t proc proc /mnt/gentoo/proc
[livecd /]# mount -o bind /dev /mnt/gentoo/dev
```

Finally, to `chroot`, execute:

```
[livecd /]# chroot /mnt/gentoo /bin/bash
[livecd /]# export PS1='${\w}chroot# '
[root@chroot]# env-update && source /etc/profile
```

Setting up the kernel

Setting up the kernel is probably the most exciting part of the whole installation process. There are more than 10 kernel sources available in Portage. We will use the recommended *gentoo-sources*, which is the slightly patched version from kernel.org. Install *gentoo-sources* with the following command:

```
[root@chroot]# emerge gentoo-sources -av
```

```
These are the packages that would be merged, in order:
Calculating dependencies... done!
```

```
[ebuild N ] sys-kernel/gentoo-sources-2.6.32-r7 USE="build -deblob
-symlink" 63,188 kB
```

```
Total: 1 package (1 new), Size of downloads: 63,188 kB
```

```
Would you like to merge these packages? [Yes/No] <type yes or RET>
<...omitted...>
```

Check that a symbolic link from `/usr/src/linux-$VERSION` to `/usr/src/linux` is created:

```
[root@chroot]# ls -ald /usr/src/linux*
lrwxrwxrwx 1 root root 25 Nov 13 2009 linux -> linux-$VERSION/
drwxr-xr-x 24 root root 4096 Jun 7 06:53 linux-$VERSION/
```

Now for the fun part: there are several ways to manually configure the kernel (without using the `genkernel`) -- *make config*, *make menuconfig*, *make xconfig* (QT-based) and *make gconfig* (GTK-based). We will choose the most popular, easy-to-use *make menuconfig*:

```
[root@chroot]# cd /usr/src/linux && make menuconfig
```

For most configurations, all you need is to select the right drivers for your system. To list all PCI devices, install `pciutils` with the command `emerge pciutils` and run `lspci`. The *make menuconfig* interface is pretty straightforward; you can either choose to build the current driver into the kernel image by pressing 'y', or build it as a

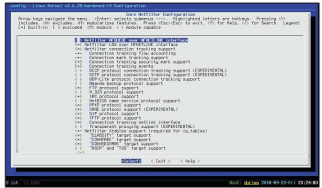


Figure 3: Kernel configuration

loadable module by pressing 'm'. Use '/' to search, and the arrow keys for navigation.

In addition to the device drivers, the following are recommended to configure a typical desktop system:

Support for filesystems:

```
EXT2_FS=y
```

```
EXT3_FS=y
```

```
CONFIG_VFAT_FS=y (for VFAT)
```

Support for USB and framebuffer console:

```
CONFIG_USB_STORAGE=y
```

```
CONFIG_FRAMEBUFFER_CONSOLE=y
```

Again, kernel configuration is the most important step; make sure all drivers for your system and file system support are configured. You should also check 'Configuring the Kernel' [4] from the Gentoo handbook.

After the configuration, we compile the kernel and the modules:

```
[root@chroot /usr/src/linux]# make -j2 && make modules_install
(this will take a while...)
```

The kernel image should be created at `/usr/src/linux/vmlinux` with the bzip-compressed version at `/usr/src/linux/arch/x86/boot/bzImage`. Copy the bziped version to `/boot`:

```
[root@chroot /usr/src/linux]# cp arch/x86/boot/bzImage /boot
```



Note: For other architectures, execute `find /usr/src/linux/arch -name bzImage` and locate your bzImage, for instance, `/usr/src/linux/x86_64/bzImage` for AMD64.

Installing and setting up the boot-loader

When a computer is powered on, the BIOS initiates tests and loads the boot-loader located on the Master Boot Record (MBR) sector of the first storage device (typically, a hard disk). The boot-loader loads the kernel, and *init*